QUICK MATHS

PRODUCT REQUIREMENTS DOCUMENT

Get better at maths.

A practical, engaging platform that helps people learn and practice the mathematics they use in everyday life—from discounts and budgeting to travel, loans, cooking, and more.

Document version	1.0
Status	Initial product blueprint for MVP planning
Primary audience	College students and Gen Z; expandable to everyone
Build approach	AI-assisted development using Google Antigravity
Design direction	Clean and minimal, with selective gamification

This document defines the initial product scope, user experience, feature requirements, technical direction, and acceptance criteria for Quick Maths. It is intended to guide implementation in small, testable stages.

Document map

1. Executive summary
2. Product vision and principles
3. Audience and user needs
4. MVP scope and exclusions
5. Topic taxonomy
6. User journeys and screens
7. Functional requirements
8. Question generation and maths correctness
9. Progress, streaks, and analytics
10. UX, visual design, and accessibility
11. Technical architecture
12. Data model
13. Privacy, security, and reliability
14. Non-functional requirements
15. Acceptance criteria
16. Development roadmap
17. Risks and mitigations
18. Open decisions and assumptions

1. Executive summary

Quick Maths is a web application for learning and practicing practical, everyday mathematics. It focuses on helping users understand the methods behind common calculations instead of relying only on a calculator. Users choose a topic, learn through concise explanations and worked examples, practice generated questions, receive feedback, and track improvement over time.

The initial audience is college students and Gen Z, with the product designed to remain useful to a broader audience. The first release prioritizes a reliable learning and practice loop, a guest-friendly entry point, persistent progress for registered users, and a polished responsive interface.

Product loop: Learn → Practice → Understand feedback → Track progress → Return.

2. Product vision and principles

Vision

Help people become more confident, capable, and independent when solving the maths they encounter in real life.

Mission

Make everyday maths approachable through short lessons, useful mental calculation strategies, realistic practice questions, and visible progress.

Product principles

- **Teach reusable methods:** explain why an answer works, not only what the answer is.
- **Think before revealing:** users attempt a question before the answer and explanation are shown.
- **Accuracy before spectacle:** reliable mathematical logic matters more than animations or gamification.
- **Low-friction start:** guests can explore and practice without creating an account.
- **Progress is broader than speed:** measure accuracy, consistency, and performance on comparable question types.
- **Expandable by design:** add topics and question templates without redesigning the whole application.

3. Audience and user needs

Primary audience

College students and Gen Z who want to improve practical maths for shopping, budgeting, travel, food, bills, and other everyday situations.

Secondary audience

Anyone who wants to strengthen everyday calculation skills, including working adults and learners returning to maths.

Core user needs

- Understand a practical maths concept in a short, clear lesson.
- Practice without feeling judged or overwhelmed.
- Receive immediate, accurate feedback and helpful explanations.
- Choose an appropriate difficulty level.
- See where they are improving and what needs more practice.
- Use the product on desktop or mobile.

• Start immediately as a guest and optionally create an account to save progress.

4. MVP scope and exclusions

In scope for MVP

• Responsive landing page and topic selection.

• Eleven topic categories with an extensible topic/subtopic structure.

• Learn pages with concepts, mental maths methods, worked examples, common mistakes, and quick checks.

• Practice flow with Easy, Medium, Hard, and Default difficulty options.

• Rule-based question generation for supported question types.

• Answer validation, hints, feedback, explanations, and session summary.

• Guest mode with limited local progress.

• Registration/sign-in and persistent progress for registered users.

• Basic progress dashboard: attempts, accuracy, average solving time, topic performance, and streak.

• Accessible responsive interface and basic error/loading/empty states.

Not required for the first release

• Public leaderboards or friend-versus-friend competition.

• A conversational AI tutor.

• AI-generated answers at runtime for core mathematical validation.

• A full budgeting, tax filing, health tracking, or financial planning product.

• Native mobile applications.

• Complex adaptive learning algorithms. Default difficulty starts at Medium and can become adaptive later.

Scope guardrail: Quick Maths teaches and practices calculations. It should not imply that its estimates replace professional financial, tax, medical, or legal advice.

5. Topic taxonomy

The topic-selection page contains these 11 top-level categories. Each category should be represented as structured data so that subtopics and lessons can be expanded independently.

#	Category	Initial subtopic examples
1	Budgeting & Expense Tracking	Monthly savings; daily expenses; household utility bills; budgeting calculations.
2	Shopping & Discounts	Single and successive discounts; net price; GST examples; bulk-buy comparisons; original price after discount.
3	Tipping & Splitting Bills	Equal and proportional splits; tips; service charges; tax-inclusive bill calculations.
4	Cooking & Recipe Scaling	Fractions; measurement conversions; scaling ingredients up or down for different servings.
5	Loans, EMI & Interest	EMI estimates; simple and compound interest; repayment totals; educational credit-card-interest examples.
6	Travel & Commute Estimation	Mileage; fuel consumption and cost; journey duration; trip cost-sharing; simple travel estimates.
7	Time & Schedule Management	Task duration; sequences of tasks; countdown calculations; time-zone differences.
8	Fitness & Health Calculations	Numerical exercises for calories, protein targets, and heart-rate zones, with clear caveats and no medical claims.

#	Category	Initial subtopic examples
9	Home Repair & Interior Work	Area; paint quantity estimates; flooring tiles; curtain fabric and measurement calculations.
10	Salary & Tax Planning	Take-home salary examples; deductions; tax estimates using clearly stated assumptions and applicable rules.
11	Ratios & Proportions	Ratios; direct proportions; sharing amounts; scaling quantities; unit rates.

Topic order can be refined based on user testing. Financial, tax, and health-related questions must state their assumptions and use appropriate disclaimers. Time-sensitive rules, such as tax slabs, should not be hard-coded without a plan for maintenance.

6. User journeys and screens

6.1 Landing page

- Headline: “Get better at maths.”
- Supporting copy explains learning practical maths, using mental calculation techniques, and tracking progress.
- Primary action: Start Practicing. Secondary action: Explore Topics.
- Show a clear preview of topic categories and how the learning loop works.

6.2 Topic selection

Display the 11 categories as responsive cards. Each card includes a name, concise description, icon, and progress indicator when data is available. Topic cards must remain understandable without relying on iconography alone.

6.3 Topic landing page

Show topic overview, available subtopics, and two clear choices: Learn and Practice. Users should be able to return to topic selection without losing session progress.

6.4 Learn experience

- Present one concept at a time using concise explanations.
- Include at least one worked example with a clear step-by-step solution.
- Include a reusable mental calculation strategy where applicable.
- Show common mistakes and a short knowledge check.

6.5 Practice difficulty modal

When Practice is selected, show Easy, Medium, Hard, and Default. Default initially maps to Medium. Explain each level briefly and allow the user to cancel or return to the topic page.

6.6 Practice question screen

- Show topic, difficulty, question text, question/session position, and answer input.
- Provide Submit Answer and Get a Hint actions.
- Do not reveal the correct answer before the first attempt, except when the user explicitly requests to reveal or end the question.
- After submission, show correct/incorrect status, the correct answer, a concise explanation, and a Continue action.
- Prevent accidental duplicate submissions and handle invalid or empty input clearly.

6.7 Session summary

Display questions attempted, correct answers, accuracy, average solving time, and a short review of the session. Offer actions to practice again, choose another topic, or view progress.

6.8 Progress dashboard

Display overall statistics, topic-wise performance, consistency/streaks, and performance trends. Use honest, understandable labels and avoid claiming improvement when the sample size is too small.

6.9 Account and guest flow

Guests can practice without registering. Registered users can save progress to an account and access it across devices. When a guest signs up, offer a safe way to merge eligible local progress without silently overwriting existing account data.

7. Functional requirements

ID	Feature	Requirement
FR-01	Navigation	Users can navigate between Home, Explore Topics, Daily Challenge (later release), Progress, and Profile. MVP may show Daily Challenge as “Coming soon” or omit it until implemented.
FR-02	Topic discovery	Users can browse all 11 categories and open a category page.
FR-03	Learn	Every published MVP subtopic has a lesson with a concept explanation and worked example; applicable lessons include a mental method and quick check.
FR-04	Difficulty selection	Practice offers Easy, Medium, Hard, and Default. Default maps to Medium in MVP.
FR-05	Question generation	The application generates valid questions from defined templates and supported value ranges for the selected topic and difficulty.
FR-06	Answer checking	Answers are validated deterministically, with explicit handling for rounding, units, currency, and accepted equivalent formats.
FR-07	Hints and explanations	A hint gives partial guidance; an explanation shows a reliable method and final result.
FR-08	Session tracking	The application tracks attempts, correctness, and elapsed solving time for a session.
FR-09	Session summary	Users see accurate summary statistics when a session ends.
FR-10	Guest mode	Guests can access published lessons and practice; limited progress can be stored locally.
FR-11	Authentication	Users can register, sign in, and sign out securely.
FR-12	Persistent progress	Registered users' attempts and progress persist across sessions and devices.
FR-13	Progress dashboard	Users can review overall and topic-level metrics.
FR-14	Streaks	A streak is based on a clearly defined practice-day rule and a consistent timezone policy.
FR-15	Responsive layout	Core flows work on common desktop, tablet, and mobile viewport sizes.
FR-16	Error states	Loading, empty, offline, invalid input, and server failure states are handled clearly.

8. Question generation and maths correctness

8.1 Deterministic question engine

Practice questions should be generated from code-defined or data-defined templates. The generator selects valid values, computes the correct answer using deterministic logic, and supplies an explanation based on the same calculation. Runtime AI should not be the authority for correctness.

8.2 Template structure

- Topic and subtopic identifier.
- Question pattern and parameter ranges.

- Difficulty level and any constraints on values.
- Correct-answer calculation.
- Accepted answer formats and rounding policy.
- Hint and explanation template.
- Validation tests and examples.

8.3 Difficulty guidance

Level	Guideline
Easy	Friendly numbers, one main operation, minimal reading load.
Medium	Less obvious numbers, common real-life scenarios, or two-step calculations.
Hard	Multi-step reasoning, reverse calculations, comparisons, or less convenient numbers.
Default	Medium for MVP; adaptive selection is a future enhancement.

8.4 Validation rules

- Test each generator across its supported ranges and edge cases.
- Avoid invalid outputs such as negative fuel consumption or impossible serving counts unless explicitly part of a lesson.
- State rounding conventions for money, percentages, mileage, and EMI.
- For currency, allow reasonable formatting differences such as commas and optional currency symbols where safe.
- For units, make the expected unit explicit and do not silently treat incompatible units as equivalent.
- Ensure hints and explanations match the generated question's actual values.
- Avoid repeating the exact same question too frequently within a session.

Example: A 25% discount on ■2,400 is ■600, so the final price is ■1,800. The question, expected answer, hint, and explanation must all use the same generated values.

9. Progress, streaks, and analytics

Core metrics

Metric	Definition / rule
Questions attempted	Count of submitted answers; define separately whether skipped questions count.
Correct answers	Count of attempts marked correct by the deterministic validator.
Accuracy	Correct answers divided by submitted attempts, expressed as a percentage; show N/A when there are no attempts.
Average solving time	Average time from question display to first submitted answer; exclude long idle periods where feasible and document the rule.
Topic performance	Attempts, accuracy, and recent activity grouped by topic and difficulty.
Current streak	Consecutive local calendar days with at least one qualifying practice activity, using a documented timezone policy.
Longest streak	Highest consecutive-day streak recorded for the user.
Trend	Performance across comparable periods or question types; avoid overstating conclusions from small samples.

Progress rules

- Guest progress may be local and device-specific; explain this limitation where appropriate.
- Registered users' progress should be stored server-side and associated with the authenticated user.
- A streak should update once per qualifying calendar day and should not increment multiple times on the same day.
- Store enough attempt data to calculate dashboard summaries and support review history.
- Do not rank users publicly in the MVP.

10. UX, visual design, and accessibility

Visual direction

Use a clean, minimal base with white/off-white surfaces, dark readable typography, green accents, restrained shadows, and consistent spacing. Use playful elements selectively in practice feedback, progress indicators, streaks, and achievements.

Interaction principles

- Make the next best action visually obvious on every screen.
- Keep the practice screen focused on one question at a time.
- Use concise feedback that is encouraging but not patronizing.
- Use animation sparingly and respect reduced-motion preferences.
- Do not communicate correctness, difficulty, or status through color alone.
- Provide keyboard-friendly controls, visible focus states, labels, and accessible error messages.
- Ensure text contrast and responsive layouts remain usable on mobile.

Voice and tone

Friendly, clear, practical, and encouraging. Explanations should use everyday language, define unfamiliar terms, and show the reasoning in small steps.

11. Technical architecture

The stack below is the recommended starting point, subject to confirmation during implementation. Keep the architecture simple and avoid unnecessary services before the MVP is validated.

Layer	Recommended choice	Purpose
Frontend	React	Reusable UI components and interactive user flows.
Language	TypeScript	Type checking and maintainable application code.
Styling	Tailwind CSS	Consistent responsive styling and design tokens.
Backend / database	Supabase + PostgreSQL	Persist users, attempts, progress, lessons, and related records.
Authentication	Supabase Auth	Secure registration and sign-in.
Hosting	Vercel or suitable equivalent	Deploy the web application.
Version control	Git + GitHub	Track changes, review diffs, and recover known-good versions.
AI-assisted development	Google Antigravity	Generate and edit code, plan tasks, and assist with testing.

Architecture guidelines

- Separate UI components, domain logic, question templates, data access, and authentication concerns.
- Keep mathematical calculations in tested, deterministic functions independent of UI components.
- Use environment variables for secrets and never commit credentials to source control.
- Use database migrations or versioned schema changes rather than undocumented manual edits.
- Enable row-level security for user-owned data and verify policies before launch.
- Use seeded demo data only when clearly separated from real user data.
- Build and test one feature slice at a time rather than generating the entire product in a single prompt.

12. Data model

Initial conceptual data model. Exact columns and relationships should be finalized during implementation and reviewed against the chosen Supabase setup.

Entity	Purpose
profiles	User profile and preferences; linked to authentication identity.
topics	Top-level categories, display order, description, icon, and publication status.
subtopics	Subtopic hierarchy within a category.
lessons	Lesson content, worked examples, methods, and quick checks.
question_templates	Template definitions, difficulty, parameter rules, answer logic identifier, and explanation pattern.
practice_sessions	User or guest session metadata, topic, difficulty, start/end time, and summary.
question_attempts	Question snapshot, submitted answer, correctness, elapsed time, and attempt timestamp.
topic_progress	Optional derived or cached topic-level summaries; source attempts remain authoritative.
daily_activity	Qualifying practice dates used for streak calculation.

Data integrity notes: store a question snapshot with each attempt so history remains understandable if a template changes. Avoid storing unnecessary personal information. Where possible, calculate summaries from authoritative attempts or update cached summaries safely.

13. Privacy, security, and reliability

- Collect only the data needed for accounts, progress, and product operation.
- Protect user-owned database rows with appropriate access policies; users must not be able to read or modify another user's private progress.
- Validate inputs on the server for persisted actions; do not rely only on client-side checks.
- Keep secrets in environment variables and out of frontend bundles and Git history.
- Provide clear sign-out behavior and a documented approach for account/data deletion before public launch.
- Do not expose private user progress in URLs, logs, public dashboards, or analytics events.
- Treat financial, tax, and health calculations as educational estimates with explicit assumptions and appropriate disclaimers.
- Provide clear error messages without leaking internal implementation details.

14. Non-functional requirements

Area	Requirement
Usability	A new user should be able to start a practice session with minimal navigation and no mandatory sign-up.
Responsive design	Core flows should work on current desktop and mobile browsers.
Performance	Common screens should feel responsive; avoid unnecessary blocking requests and oversized assets.
Correctness	Every shipped question template must have automated tests for answer calculation, constraints, and explanation consistency.
Reliability	Handle refreshes, network failures, and interrupted sessions gracefully; clearly indicate what has or has not been saved.
Accessibility	Keyboard access, visible focus, semantic structure, readable contrast, labels, and reduced-motion support.
Maintainability	Use consistent naming, modular code, documented calculations, and version control.
Privacy	Collect minimal personal data and restrict access to user-owned records.
Observability	Log useful technical errors without exposing passwords, tokens, or unnecessary personal data.

15. Acceptance criteria

ID	Acceptance criterion
AC-01	A visitor can open the landing page and navigate to all published topic categories.
AC-02	The topic page shows all 11 categories defined in this PRD.
AC-03	A user can choose Learn or Practice from a topic page.
AC-04	Practice offers Easy, Medium, Hard, and Default; Default maps to Medium in the MVP.
AC-05	A generated question has a valid answer, matching hint, and matching explanation.
AC-06	The user can submit an answer and receive correct/incorrect feedback without duplicate submissions.
AC-07	The session summary accurately reflects submitted attempts, correctness, and solving times.
AC-08	A guest can practice without creating an account and can see the current session summary.
AC-09	A registered user can sign in and retrieve saved progress on another supported device.
AC-10	A user cannot access another user's private progress by changing a request or identifier.
AC-11	The progress dashboard handles the zero-attempt state without showing misleading percentages.
AC-12	Core flows are usable at desktop and mobile viewport sizes and with keyboard navigation.
AC-13	Automated tests cover each released calculation template and key edge cases.
AC-14	Errors, loading states, and unavailable network states are handled with clear user feedback.

16. Development roadmap

Phase	Deliverables
Phase 0 — Project setup	Create repository, app skeleton, code conventions, environment setup, and basic build/test workflow.
Phase 1 — Design foundation	Define typography, colors, spacing, buttons, cards, navigation, and responsive page structure.

Phase	Deliverables
Phase 2 — Topic discovery	Build landing page, 11 topic cards, topic detail pages, and Learn/Practice choices.
Phase 3 — Learn experience	Add initial lessons and worked examples for the first set of topics.
Phase 4 — Practice engine	Implement question templates, difficulty selection, validation, hints, explanations, and session summaries.
Phase 5 — Accounts and progress	Integrate authentication, persistent attempts, dashboard metrics, and streak logic.
Phase 6 — QA and launch	Test calculations, security policies, responsive behavior, accessibility, error handling, and deployment.
Later — Engagement	Daily challenge, achievements, improved adaptive difficulty, broader topic coverage, and optional social features.

Recommended implementation style: ask Antigravity to complete one small, testable phase or feature at a time. Review the code changes, run the application, test the acceptance criteria, and commit a working version before moving on.

17. Risks and mitigations

Risk	Mitigation
Incorrect generated answers	Use deterministic calculation functions, template tests, and edge-case coverage; never rely on a language model as the answer validator.
Scope grows too quickly	Keep the MVP focused on the core learn/practice/progress loop; defer leaderboards, AI tutor, and advanced adaptive logic.
Lessons feel repetitive	Use varied question patterns and practical scenarios; validate quality with real users.
Progress metrics mislead users	Define metrics explicitly, compare like with like, and show uncertainty when there are too few attempts.
Sensitive-topic calculations become outdated	State assumptions, isolate rules that may change, and review financial/tax/health content before release.
AI-generated code is difficult to maintain	Use small prompts, inspect diffs, run tests, keep modular architecture, and commit known-good changes.
Guest progress is lost	Explain that guest data is device-specific and design a safe account-upgrade/merge flow.
Mobile experience is weak	Test core journeys at mobile sizes from the beginning rather than postponing responsive work.

18. Open decisions and assumptions

Decisions already established

- Product name: Quick Maths.
- Tagline: “Get better at maths.”
- Primary audience: college students and Gen Z, with wider applicability.
- Account model: guest access plus optional registration for synchronized progress.
- Practice feel: clean and focused with optional gamification.
- Visual style: clean and minimal, with selected playful elements.
- Initial topic categories: the 11 categories listed in Section 5.
- Default difficulty: Medium for the MVP; adaptive behavior is deferred.
- Question validation: deterministic mathematical logic.

Assumptions to confirm during implementation

- Initial release is a responsive web application, not a native mobile app.
- English is the first interface language; localization can be added later.
- The MVP will launch with a carefully selected subset of subtopics and expand after testing.
- Daily Challenge may be deferred until the core practice flow is stable.
- A specific retention and deletion policy for user data will be decided before public launch.
- The exact hosting, authentication configuration, database schema, and pricing should be confirmed before production deployment.

Definition of MVP success

The MVP is ready for an initial user test when people can discover a topic, understand a lesson, answer a generated question accurately, learn from feedback, finish a session, and review their progress—with no critical calculation, authentication, or data-access defects.

End of document • Quick Maths PRD v1.0